SAMSUNG

Samsung Innovation Campus

Generative AI Module

Together for Tomorrow!
Enabling People
Education for Future Generations

Notas de Clase

Generative AI Module

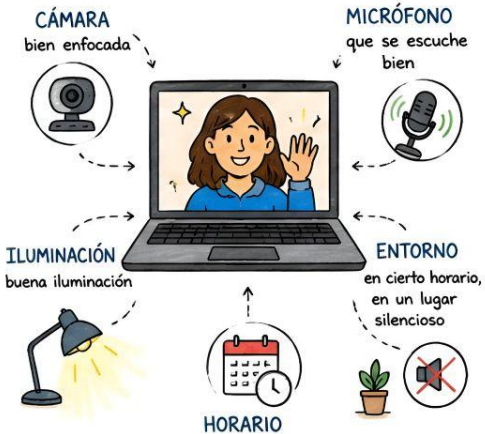
OBJETIVOS DEL CURSO

1 Se lleven como razonar un problema y saber cuando usar AI y cuando no.



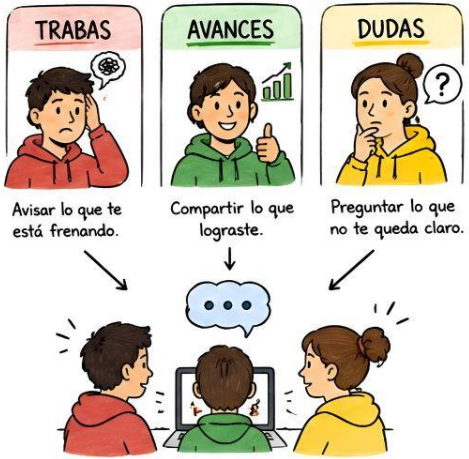
Primero pensar, después decidir.

2 Organizar su vida para tener meets, eso implica tener tu carta de presentacion en buenas condiciones:



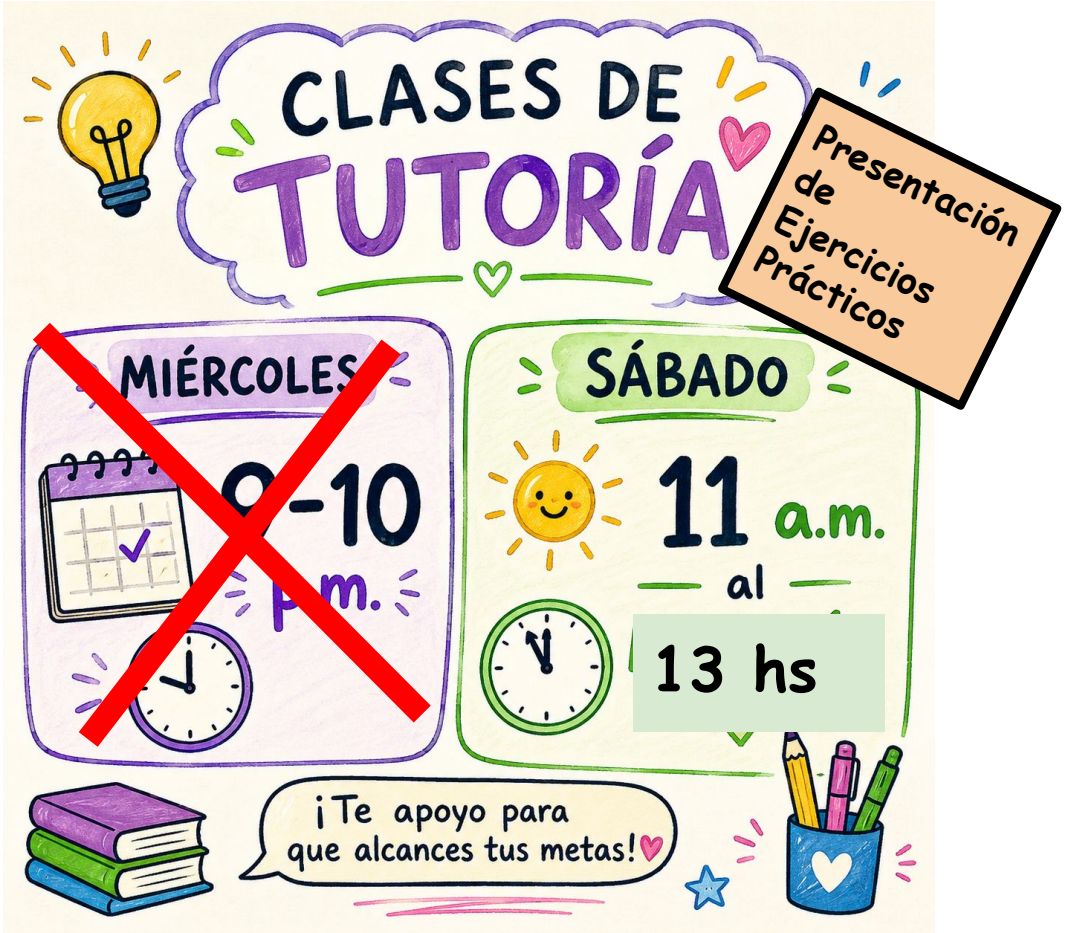
Tu meet es tu carta de presentación. ¡Cuidala!

3 Poder comunicar trabas, avances y dudas a tiempo, es preferible equivocarse aca que con una deadline.



Comunicar a tiempo = mejores soluciones, menos estrés.

PENSAR. ORGANIZAR. COMUNICAR. = CRECER JUNTOS



[Inscripción a la reunión - Zoom]

CLASE ANTERIOR

OBJETIVO: Construir un dataset de libros de terror a partir de datos de la web, limpiarlo y responder preguntas sobre disponibilidad, similitud y precios.

1. REVISIÓN RÁPIDA

- 2.3 Operaciones con Series
- 2.4 Operaciones con DataFrames
- 2.5 Combinar datos (concat / merge / join)



2. PIPELINE GENERAL

APIFY → Scraping → {;} JSON / CSV → PYTHON + REQUESTS → ETL

- extracción
- limpieza
- normalización
- validación

→ books_terror_clean.csv

3. ETL BÁSICO

EXTRACT Obtener datos de Apify → {...}

TRANSFORM Limpiar y normalizar

Título	Precio
" Drácula " DRÁCULA drácula " → drácula	"$ 15.999 " "15.999,00" "ARS 15999" "Precio no disp." "Sin stock" NULL " → 15999.0

LOAD Guardar dataset limpio → CSV

4. BÚSQUEDA PDF

¿Existe el PDF del libro?



title	pdf_found	pdf_url
Drácula	✓	https://...
El exorcista	✗	—
It	✓	https://...

Discutimos límites legales y éticos de las búsquedas.

5. SIMILITUD SEMÁNTICA

¿Qué libros son similares entre sí?

TÍTULO + AUTOR + DESCRIPCIÓN → EMBEDDINGS → COSINE SIMILARITY

Top pares más similares

book_a	book_b	similarity
Drácula	Carmilla	0.87
It	La llamada de Cthulhu	0.84
El exorcista	El resplandor	0.81
...	...	...

6. SEGUNDO EXPERIMENTO: ELEGIR UN TÍTULO Y OBTENER 50 REGISTROS

Ejemplo: DRÁCULA
50 publicaciones



title	seller	price	original_price	discount	date
Drácula	Vendedor A	12000	15000	20%	18/08/26
Drácula	Vendedor B	14500	—	0%	18/08/26
Drácula	Vendedor C	10999	13999	21%	18/08/26
...	...	...			

7. ANÁLISIS DE PRECIOS Y DESCUENTOS

Distribución de precios

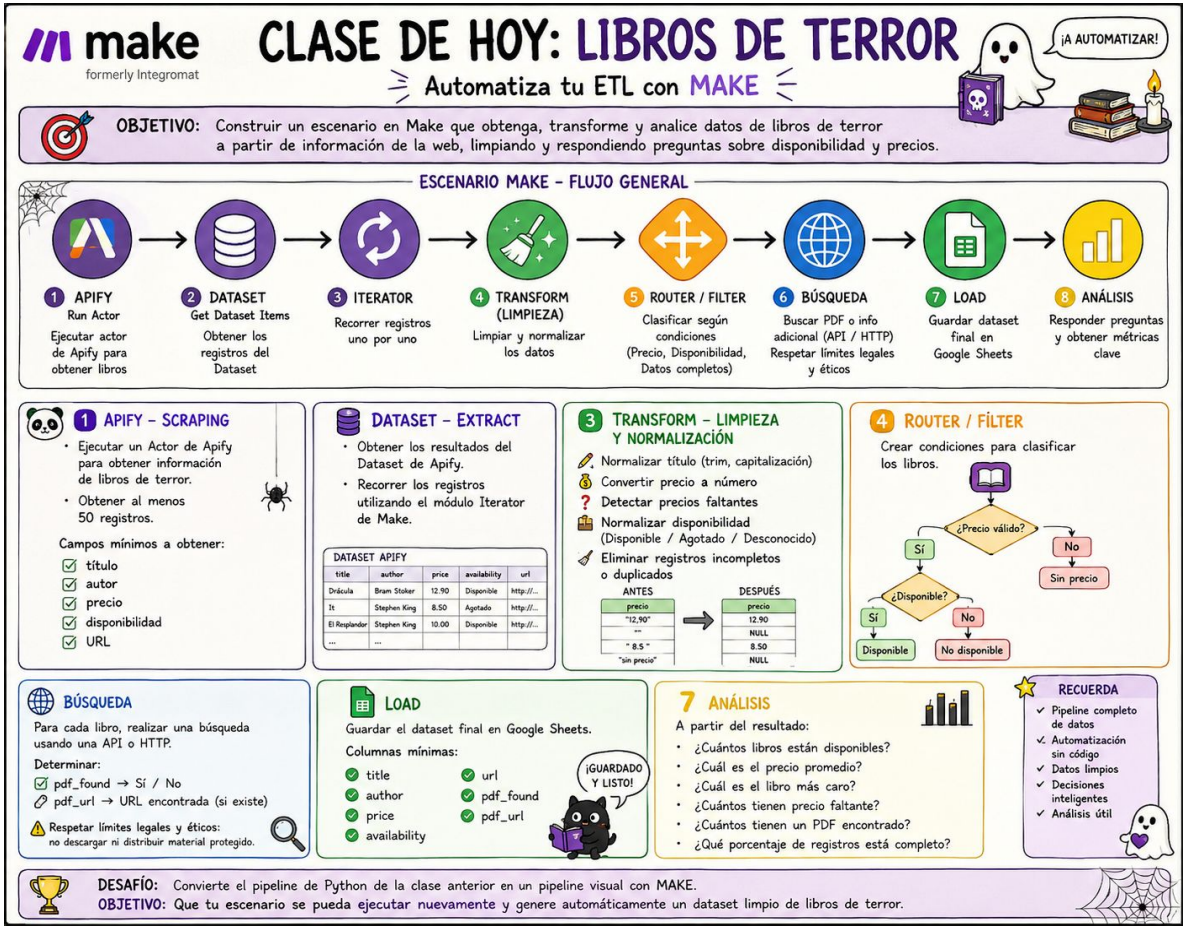


¿Tuvo descuentos?

has_discount = (discount > 0)

Comparar precios con vs sin descuento




make
formerly Integromat
CLASE DE HOY: LIBROS DE TERROR
Automatiza tu ETL con MAKE
¡A AUTOMATIZAR!
OBJETIVO: Construir un escenario en Make que obtenga, transforme y analice datos de libros de terror a partir de información de la web, limpiando y respondiendo preguntas sobre disponibilidad y precios.
ESCENARIO MAKE - FLUJO GENERAL
1 APIFY
Run Actor
Ejecutar actor de Apify para obtener libros
2 DATASET
Get Dataset Items
Obtener los registros del Dataset
3 ITERATOR
Recorrer registros uno por uno
4 TRANSFORM (LIMPIEZA)
Limpiar y normalizar los datos
5 ROUTER / FILTER
Clasificar según condiciones (Precio, Disponibilidad, Datos completos)
6 BÚSQUEDA
Buscar PDF o info adicional (API / HTTP) Respetar límites legales y éticos
7 LOAD
Guardar dataset final en Google Sheets
8 ANÁLISIS
Responder preguntas y obtener métricas clave
1 APIFY – SCRAPING
Ejecutar un Actor de Apify para obtener información de libros de terror.
Obtener al menos 50 registros.
Campos mínimos a obtener:
título
autor
precio
disponibilidad
URL
DATASET – EXTRACT
Obtener los resultados del Dataset de Apify.
Recorrer los registros utilizando el módulo Iterator de Make.
DATASET APIFY
title author price availability url
Drácula Bram Stoker 12.90 Disponible http://...
It Stephen King 8.50 Agotado http://...
El Resplandor Stephen King 10.00 Disponible http://...
3 TRANSFORM – LIMPIEZA Y NORMALIZACIÓN
Normalizar título (trim, capitalización)
Convertir precio a número
Detectar precios faltantes
Normalizar disponibilidad (Disponible / Agotado / Desconocido)
Eliminar registros incompletos o duplicados
ANTES
precio
"12,90"
""
" 8.5 "
"sin precio"
DESPUÉS
precio
12.90
NULL
8.50
NULL
4 ROUTER / FILTER
Crear condiciones para clasificar los libros.
¿Precio válido?
Sí
No
Sin precio
¿Disponible?
Sí
No
Disponible
No disponible
BÚSQUEDA
Para cada libro, realizar una búsqueda usando una API o HTTP.
Determinar:
pdf_found → Sí / No
pdf_url → URL encontrada (si existe)
Respetar límites legales y éticos: no descargar ni distribuir material protegido.
LOAD
Guardar el dataset final en Google Sheets.
Columnas mínimas:
title
author
price
availability
url
pdf_found
pdf_url
¡GUARDADO Y LISTO!
7 ANÁLISIS
A partir del resultado:
¿Cuántos libros están disponibles?
¿Cuál es el precio promedio?
¿Cuál es el libro más caro?
¿Cuántos tienen precio faltante?
¿Cuántos tienen un PDF encontrado?
¿Qué porcentaje de registros está completo?
RECUERDA
Pipeline completo de datos
Automatización sin código
Datos limpios
Decisiones inteligentes
Análisis útil
DESAFÍO: Convierte el pipeline de Python de la clase anterior en un pipeline visual con MAKE.
OBJETIVO: Que tu escenario se pueda ejecutar nuevamente y genere automáticamente un dataset limpio de libros de terror.

Dataset Filter & Transform

from $2.00 / 1,000 row kepts | Limited permissions | Start | Save as a new task | API | ...

nerolabs/dataset-filter-transform | 0.0 (0) | 0 | 16 | 12 | Crafted by Adam Pearce

Returns rows of any Apify dataset, CSV/Excel/JSON file URL or Google Sheet that pass filter rules (25 operators, AND/OR) after transforms (rename, cast, compute, regex, dates, replace, split, hash), then sort, dedupe, limit, export CSV/Excel or webhook. Agent-ready: pay per event (x402, MCP).

Input | Info | Runs 1 | Builds 5 | Integrations 0 | Monitoring | Issues 0 | Tasks 0 | Reviews

Build 0.1.5 / latest | Readme | Input

Stop writing one-off scripts to clean up scraper output. Dataset Filter & Transform takes any Apify dataset, a CSV, Excel or JSON file by URL, a Google Sheet, or a JSON array you paste in, and lets you **filter rows by rule** and **transform fields**: rename, drop, trim, change case, cast types, compute a field from arithmetic, extract text with a regex, replace, split, fix dates, map values, hash a stable id, or template a new field from existing ones. Then sort, dedupe, limit, and get back exactly the rows and fields you wanted, as a dataset, a ready-to-open CSV or Excel file, a named dataset that accumulates across runs, or a POST to your webhook. All from a JSON config, no code, no spreadsheet formulas to babysit.

ACTOR METRICS

- 16 total users
- 12 monthly active users
- No reviews yet
- No bookmarks yet
- 98% runs succeeded

Google Search Results Scraper

from $4.99 / 1,000 results | Limited permissions | Save & start | Save as a new task | API | ...

scraper-engine/google-search-results-scraper | ★ 0.0 (0) | 0 | 32 | 10 | Crafted by Scraper Engine

Google Search Results Scraper extracts structured data from Google search pages in real time. Collect titles, URLs, snippets, rankings, ads, and related queries in JSON or CSV. Ideal for SEO tracking, competitor research, and keyword performance analysis.

Input | Info | Runs 0 | Builds 3 | Integrations 0 | Monitoring | Issues 0 | Tasks 0 | Reviews

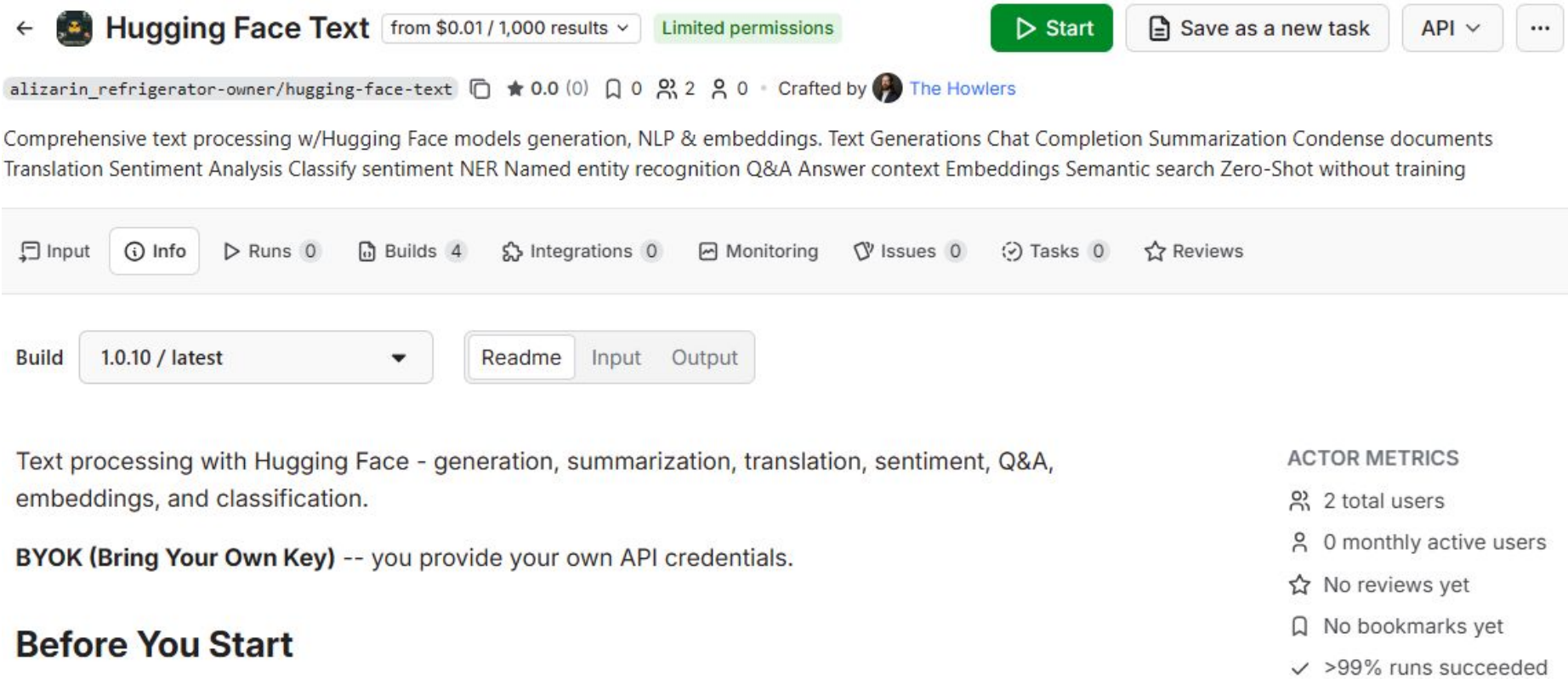
Build 0.1.7 / latest | Readme | Input

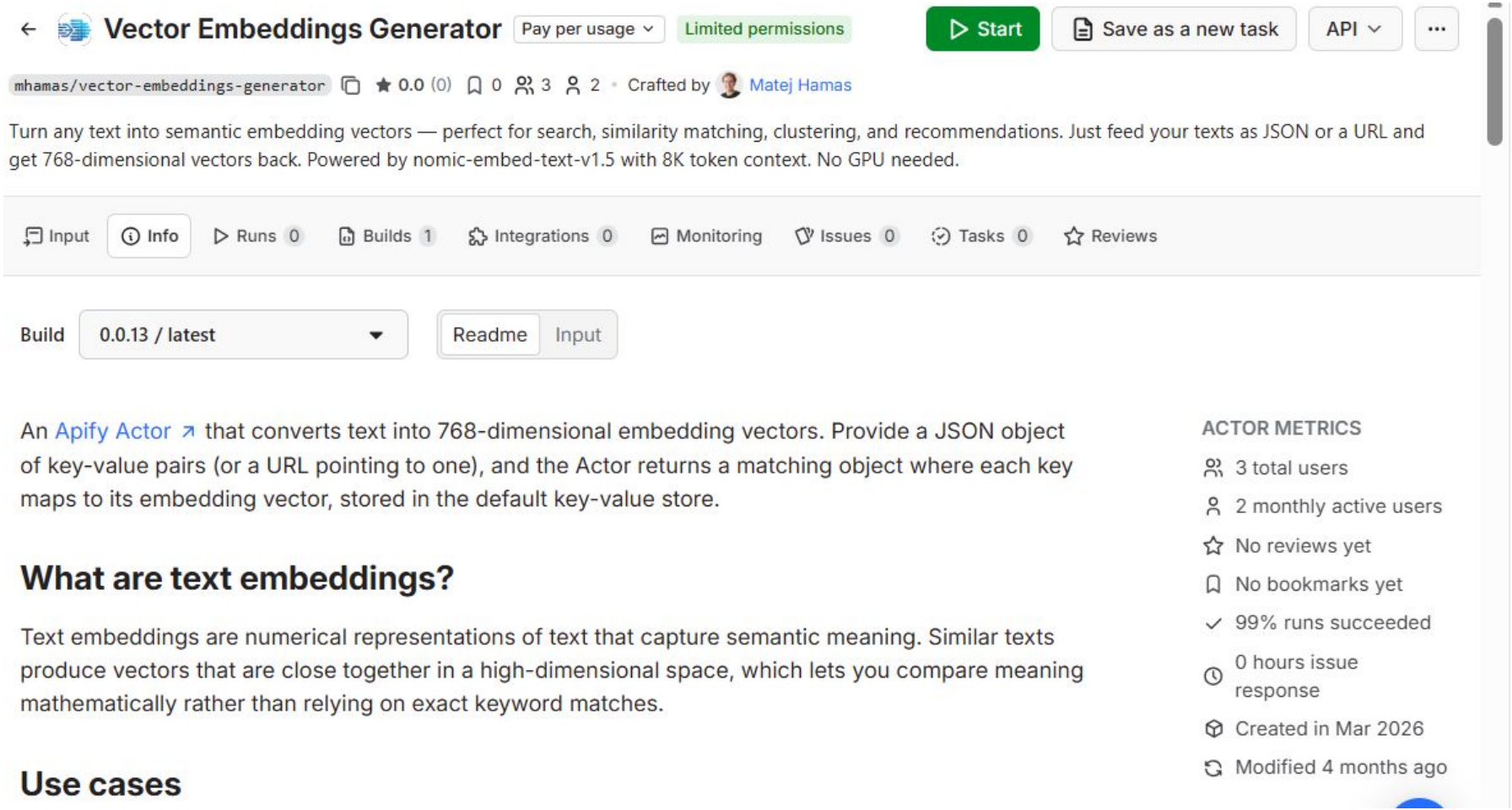
Google Search Results Scraper — Organic, Ads and AI Overview Data

Google Search Results Scraper runs real Google searches and returns organic results, paid ads, shopping products, People Also Ask questions, related searches, and an optional AI Overview as structured JSON — no HTML parsing required. Feed it plain keywords or full Google Search URLs, layer on `site:`, `intitle:`, `filetype:`, and date-range filters, and get one dataset row per page per query, ready to load into a spreadsheet, database, or LLM pipeline. Start a run from the Actor page below.

ACTOR METRICS

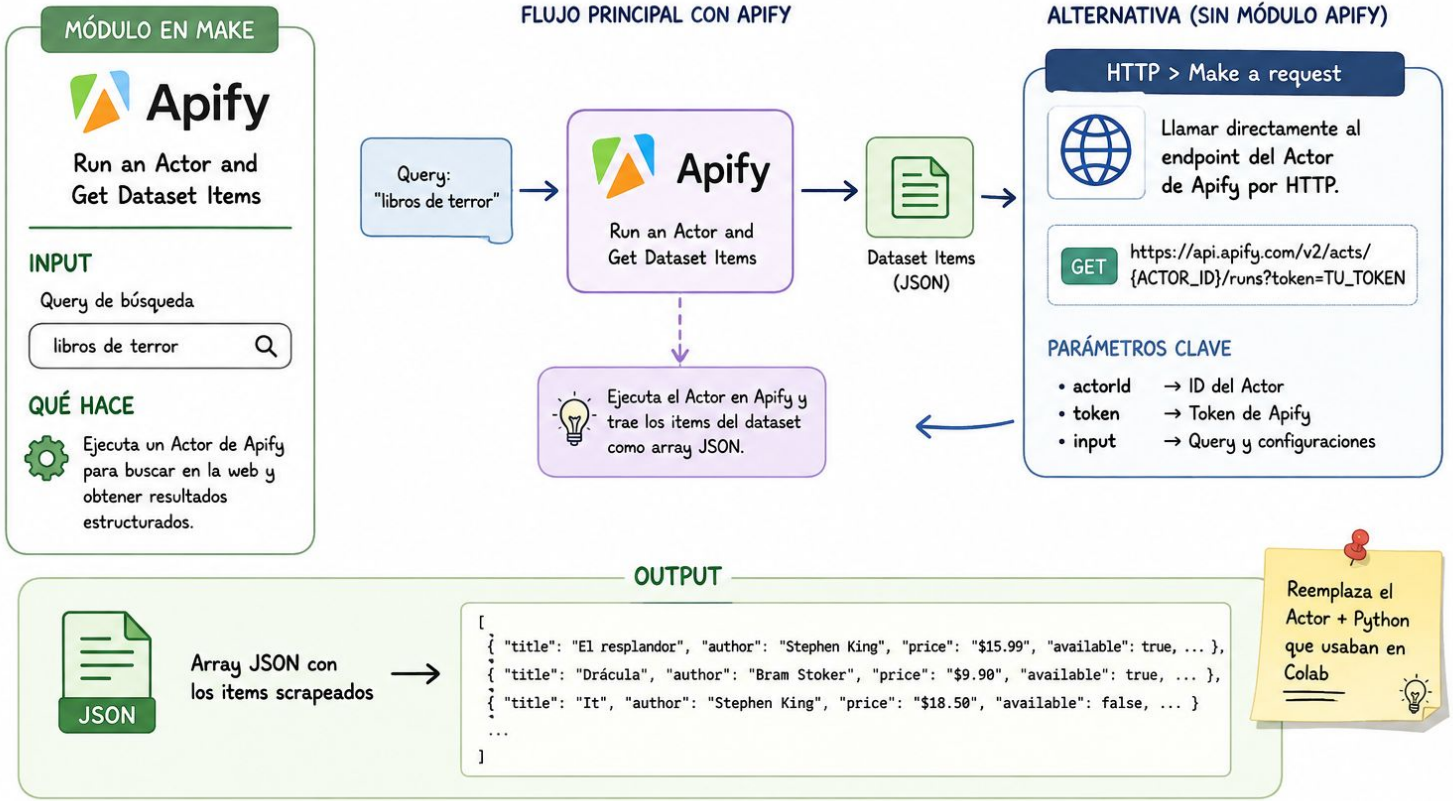
- 32 total users
- 10 monthly active users
- No reviews yet
- No bookmarks yet
- >99% runs succeeded
- 0 hours issue response


Hugging Face Text
from $0.01 / 1,000 results
Limited permissions
Start
Save as a new task
API
alizarin_refrigerator-owner/hugging-face-text
0.0 (0)
0
2
0
Crafted by
The Howlers
Comprehensive text processing w/Hugging Face models generation, NLP & embeddings. Text Generations Chat Completion Summarization Condense documents Translation Sentiment Analysis Classify sentiment NER Named entity recognition Q&A Answer context Embeddings Semantic search Zero-Shot without training
Input
Info
Runs 0
Builds 4
Integrations 0
Monitoring
Issues 0
Tasks 0
Reviews
Build
1.0.10 / latest
Readme
Input
Output
Text processing with Hugging Face - generation, summarization, translation, sentiment, Q&A, embeddings, and classification.
BYOK (Bring Your Own Key) -- you provide your own API credentials.
Before You Start
ACTOR METRICS
2 total users
0 monthly active users
No reviews yet
No bookmarks yet
>99% runs succeeded


Vector Embeddings Generator
Pay per usage
Limited permissions
Start
Save as a new task
API
mhamas/vector-embeddings-generator
0.0 (0)
0
3
2
Crafted by Matej Hamas
Turn any text into semantic embedding vectors — perfect for search, similarity matching, clustering, and recommendations. Just feed your texts as JSON or a URL and get 768-dimensional vectors back. Powered by nomic-embed-text-v1.5 with 8K token context. No GPU needed.
Input
Info
Runs 0
Builds 1
Integrations 0
Monitoring
Issues 0
Tasks 0
Reviews
Build
0.0.13 / latest
Readme
Input
An Apify Actor that converts text into 768-dimensional embedding vectors. Provide a JSON object of key-value pairs (or a URL pointing to one), and the Actor returns a matching object where each key maps to its embedding vector, stored in the default key-value store.
What are text embeddings?
Text embeddings are numerical representations of text that capture semantic meaning. Similar texts produce vectors that are close together in a high-dimensional space, which lets you compare meaning mathematically rather than relying on exact keyword matches.
Use cases
ACTOR METRICS
3 total users
2 monthly active users
No reviews yet
No bookmarks yet
99% runs succeeded
0 hours issue response
Created in Mar 2026
Modified 4 months ago

1. Scraping – Módulo Apify

Reemplaza el Actor + Python que usaban en Colab



2. ETL — 3 formas de limpiar en Make

Reemplaza las funciones de limpieza que hacían con pandas — elegí según herramientas disponibles



LIMPIEZA Y TRANSFORMACIÓN

A Code (JS)

Requiere plan Developer

Normalizá precios, descuentos, disponibilidad y texto en una sola función

```
function cleanItem(item) {
  item.precio = normalizarPrecio(item.precio);
  item.descuento = normalizarDescuento(item.descuento);
  item.disponibilidad = normalizarDisponibilidad(item.stock);
  item.titulo = normalizarTexto(item.titulo);
  return item;
}
```

Ejemplo de limpieza

ANTES (raw)	DESPUÉS (limpio)
precio: "$ 12.990,00"	precio: 12990
descuento: "-20 %"	descuento: 20
disponibilidad: "En Stock "	disponibilidad: "En stock"
titulo: " IT de Stephen King "	titulo: "IT de Stephen King"

Ventajas: todo en una función, flexible, potente

Desventajas: requiere plan Developer y conocimientos JS

B Text Parser

Encadená módulos Text parser para cada regla de transformación

Text parser > Replace Quitar símbolos de moneda y espacios	"$ 12.990,00" → "12.990,00"
Text parser > Replace Reemplazar coma decimal por punto	"12.990,00" → "12990.00"
Text parser > Match pattern (regex) Extraer número	"12990.00" → 12990
Text parser > Replace / Lowercase Normalizar disponibilidad	"En Stock " → "En stock"

Ventajas: simple de implementar, no requiere código

Desventajas: más módulos, más largo y frágil

C Excel/Sheets o Apps Script

Pasá los datos a Sheets y limpiá con fórmulas o un script de Apps Script

Google Sheets (fórmulas)

	A (raw)	B (limpio)
2	$ 12.990,00	=VALOR(SUSTITUIR(A2;" ";" "))
3	-20 %	=VALOR(SUSTITUIR(A3;"%";""))
4	En Stock "	=ESPACIOS(MAYUSC(A4))

ó

Apps Script (lógica de limpieza)

```
function limpiarFila(fila) {
fila.precio = normalizarPrecio(fila.precio);
fila.descuento = normalizarDescuento(fila.descuento);
fila.disponibilidad = normalizarDisponibilidad(fila.stock);
fila.titulo = normalizarTexto(fila.titulo);
return fila;
}
```

Ventajas: muy flexible, escalable, sin límite de módulos en Make

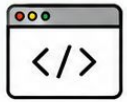
Desventajas: depende de Sheets / permisos / mantenimiento

Idea clave: Elegí la opción que mejor se adapte a tus herramientas y plan. | Code (JS) = más control | Text Parser = más simple | Sheets/Apps Script = más escalable y flexible

3. Búsqueda PDF — 3 formas de extraer datos

Reemplaza *requests* + BeautifulSoup de Python



1. Buscar en la web → **HTTP** > Make a request GET a DuckDuckGo HTML →  HTML recibido → **2. Extraer link PDF y snippet** →  Link PDF Snippet

INPUT

libros de terror pdf

Ejemplo de URL generada:

https://duckduckgo.com/html/?q=libros+de+terror+pdf

OUTPUT ESPERADO

- Link PDF
- Snippet (título + descripción)

OPCIÓN A — CODE (JS)

Parsear el HTML con expresiones regulares o un mini-parser propio

```
const html = bundle.body;

// Extraer links PDF
const regex = /href="(https?:[^"']+?\.pdf)"/gi;
const match = html.match(regex);
const pdfLinks = [...new Set(match || [])];

// Extraer snippet (ej. primera descripción)
const snippetMatch = html.match(/<a[^>]+>(.*?)</a>
\s*<\/a>[\t\n\r]*(\/a>)/s);
const snippet = snippetMatch ?
  snippetMatch[1].replace(/\s+/g, ' ').trim() : '';
```

- **PROS:** Flexible y potente.
- **CONTRAS:** Requiere plan Developer y conocimiento de JS.

OPCIÓN B — TEXT PARSER

Usar módulos de Text Parser con expresiones regulares

EJEMPLO DE REGEX

1) Match pattern (link PDF)

`href="(https?:[^"']+?\.pdf)"`

→ Grupo 1 = URL del PDF

2) Match pattern (snippet)

`<a[^>]*class="result__a"[^>]*>(.*?)</a>`

→ Grupo 1 = Título del resultado

- **PROS:** No requiere código. Muy visual.
- **CONTRAS:** Más frágil si cambia el HTML. Múltiples módulos para reglas complejas.

OPCIÓN C — EXCEL / APPS SCRIPT

Hacer el scraping fuera de Make con Apps Script y funciones de texto

EJEMPLO APPS SCRIPT

```
function buscarPDF(query) {
  const url = 'https://duckduckgo.com/html/?q=' +
    encodeURIComponent(query);
  const html = UrlFetchApp.fetch(url).getContentText();
  const regex = /href="(https?:[^"']+?\.pdf)"/gi;
  const matches = html.match(regex) || [];
  const pdfLink = matches[0] ? matches[0].match(regex)[1] : '';

  // Extraer snippet (simplificado)
  const snippetMatch = html.match(/<a[^>]*class="result__a"[^>]*>
  (.*?)<\/a>/s);
  const snippet =
  snippetMatch ? snippetMatch[1].replace(/\s+/g, ' ').trim() : '';
  return { pdfLink, snippet };
}
```

- **PROS:** Más control y reutilizable en Sheets.
- **CONTRAS:** Requiere escribir script (Apps Script).

TIP: DuckDuckGo HTML no tiene API oficial. Los selectores/regex pueden romperse si cambia el sitio. Para producción, considerar APIs de búsqueda o proveedores especializados.



Idea clave: las 3 opciones permiten reemplazar BeautifulSoup. Elegí la que mejor se adapte a tus herramientas y nivel de control.

4. Embeddings y Similitud — HTTP + Code

Objetivo: convertir textos en vectores (embeddings) y medir cuán similares son.

1. TEXTOS



Títulos / Resúmenes de libros

2. HTTP → API de Embeddings

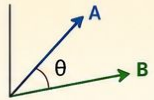


HuggingFace / OpenAI / Microservicio propio (aiserver)

3. EMBEDDINGS (VECTORES)

0.12	−0.35	...	0.78	(libro 1)
−0.21	0.47	...	−0.11	(libro 2)
⋮				
0.05	−0.08	...	0.31	(libro n)

4. SIMILITUD (COSENO)



similaridad = cos(θ)

CLAVE

No hay sentence-transformers nativo en Make.

La generación de embeddings **siempre se hace por HTTP** a un servidor externo.

→ **Delegar el cómputo pesado a un servidor externo.**

OPCIÓN A — CODE (JS)

Cálculo de similitud coseno en Make

```
function cosineSimilarity(vecA, vecB) {
  let dot = 0;
  let normA = 0;
  let normB = 0;

  for (let i = 0; i < vecA.length; i++) {
    dot += vecA[i] * vecB[i];
    normA += vecA[i] * vecA[i];
    normB += vecB[i] * vecB[i];
  }

  return dot / (Math.sqrt(normA) * Math.sqrt(normB));
}
```

Uso:

- Recibe dos arrays de números (embeddings)
- Devuelve un valor entre -1 y 1 (1 = idénticos, 0 = sin relación, -1 = opuestos)

JS

OPCIÓN B — EXCEL / GOOGLE SHEETS

Fórmulas para coseno

Supongamos los vectores en filas:

	A	A	B	C	D	... N
1	Vec A	0.12	−0.35	...	0.78	
2	Vec B	−0.21	0.47	...	−0.11	

1. Producto punto (A · B):
`=SUMAPRODUCTO(A1:N1, A2:N2)`

2. Norma de A:
`=RAIZ(SUMAPRODUCTO(A1:N1, A1:N1))`

3. Norma de B:
`=RAIZ(SUMAPRODUCTO(A2:N2, A2:N2))`

4. Similitud coseno:
`=(A · B) / (NormaA * NormaB)` → Resultado: **0.842**

OPCIÓN C — APPS SCRIPT

Función custom en Google Sheets

```
function COSENO(v1, v2) {
  var dot = 0, normaA = 0, normaB = 0;
  for (var i = 0; i < v1.length; i++) {
    dot += v1[i] * v2[i];
    normaA += v1[i] * v1[i];
    normaB += v2[i] * v2[i];
  }
  return dot / (Math.sqrt(normaA) * Math.sqrt(normaB));
}
```

Cómo usar:

1. Extensiones > Apps Script
2. Pegar la función
3. Usar en hoja: =COSENO(A1:N1, A2:N2)

Resumen: Embeddings vía HTTP + Similitud con Code, Sheets o Apps Script. → Flexibilidad para elegir según tus herramientas y necesidades.

5. Estadística — 3 formas de calcular

Calcular métricas básicas: media, desviación estándar y percentiles



OPCIÓN A — GOOGLE SHEETS

Usar fórmulas nativas en Google Sheets

	A	B	C
	Precio	Descuento (%)	Disponibles
2	1200	10	25
3	890	0	12
4	1500	15	30
5	750	5	18

FÓRMULAS EJEMPLO

- =PROMEDIO(A2:A5) → Media
- =DESVEST.M(A2:A5) → Desv. estándar
- =PERCENTIL.INC(A2:A5;0,5) → Mediana (P50)
- =PERCENTIL.INC(A2:A5;0,9) → P90

OPCIÓN B — CODE (JS)

Funciones de estadística escritas en JavaScript

```
function mean(arr) {
  return arr.reduce((a,b) => a + b, 0) / arr.length;
}

function std(arr) {
  const m = mean(arr);
  const v = arr.reduce((s, x) => s + Math.pow(x - m, 2)
            / (arr.length - 1);
  return Math.sqrt(v);
}

function percentile(arr, p) {
  const sorted = [...arr].sort((a,b) => a - b);
  const idx = (sorted.length - 1) * p;
  const lower = Math.floor(idx);
  const upper = Math.ceil(idx);
  if (lower === upper) return sorted[lower];
  return sorted[lower] + (sorted[upper] - sorted[lower])
         * (idx - lower);
}
```

Media, Desv. Est, Percentiles

Mayor control y flexibilidad.
Ideal para lógica personalizada.

OPCIÓN C — APPS SCRIPT

Automatizar cálculos en Sheets con Google Apps Script

```
function calcularEstadisticas() {
  const sh = SpreadsheetApp.getActiveSpreadsheet()
             .getSheetByName('Datos');
  const datos = sh.getRange('A2:A').getValues()
                .flat().filter(String);

  const media = mean(datos);
  const desvest = std(datos);
  const p50 = percentile(datos, 0.5);
  const p90 = percentile(datos, 0.9);

  sh.getRange('D2:D5').setValues([
    ['Media', media],
    ['Desv. Est.', desvest],
    ['P50', p50],
    ['P90', p90]
  ]);
}
```

Se ejecuta con trigger o manualmente

Automatización total.
Sin tocar fórmulas manualmente.

No existe numpy nativo en Make → Sheets es más rápido de implementar; Code/Apps Script dan más control y automatización



6. Output — Google Sheets / Drive / Data Store

Guardamos el resultado final del flujo para usarlo, compartirlo o reutilizarlo en otros escenarios.

Data Store es preferible para flujos que se reutilizan

Dataset limpio
(Array de items)

[{ … }, { … }, { … }]

Viene del paso anterior
(Array Aggregator)

Make — Módulo de Output
Elegí dónde guardar el dataset limpio

Opción 1 — Google Sheets
Google Sheets > Add a row
Ideal para análisis y dashboards rápidos

Opción 2 — Google Drive
Google Drive > Upload a file
Guarda CSV/Excel/JSON para compartir o descargar

Opción 3 — Make Data Store
Data Store > Add/Update a record
Persistencia interna de Make para reutilizar en otros escenarios

¿Qué obtenés?

- Tabla estructurada
- Fácil de filtrar y analizar
- Conectar con Looker Studio

CSV

- Archivo listo para descargar
- Exportable a Excel / CSV / JSON
- Compartible por link

- Reutilizable en otros escenarios
- Consultable por otros módulos
- Sin depender de servicios externos

Resultado final
Equivalente a books_terror_clean.csv generado en Python

¿Para qué se usa?

- Análisis de precios y descuentos
- Investigación de libros y tendencias
- Entrenamiento de modelos / embeddings
- Alertas y reportes automáticos

Guardá una vez, usá muchas veces.

SLIDE 7

Diseño sugerido del flujo completo

Make como orquestador • Cómputo pesado fuera • Guardado y notificaciones

Idea clave
No reimplementamos pandas, numpy ni sentence-transformers en Make. ¡Delegamos y orquestamos!

1. APIFY (SCRAPING)

Run an Actor and Get Dataset Items

APIFY

- Actor: Amazon Books
- Query: "libros de terror"
- Output: JSON (items)

→ { } JSON →

2–5. aiservers (CÓMPUTO PESADO VIA HTTP)

ETL + PDF search + Embeddings + Estadística

ETL / LIMPIEZA	BÚSQUEDA PDF	EMBEDDINGS	ESTADÍSTICA
Normaliza precios, descuentos, texto, disponibilidad, etc.	Busca y extrae link PDF y snippet (desde DuckDuckGo)	Obtiene embeddings (vía API externa) y calcula similitud coseno	Media, mediana, desvío, percentiles, distribución de precios y descuentos

aiserver (microservicio propio)
Expone un endpoint HTTP que recibe el JSON y devuelve el resultado procesado

→ { } JSON →

6. OUTPUT (GUARDADO)

Guardar resultados

- **Google Sheets** — Add a row
- **Google Drive** — Upload a file (CSV/Excel)
- **Data Store (Make)** — Para reuso en otros escenarios

→

NOTIFICACIONES

Informar resultados

- **Gmail** — Email con resumen y link al archivo
- **Slack** — Mensaje al canal
- **Webhook** — Integración con otra herramienta

ALTERNATIVA LOW-CODE (SIN aiservers)

Apify → Sheets/Apps Script → Make (solo dispara y notifica)

APIFY → **Google Sheets** Datos crudos → **Apps Script (o fórmulas)** Limpieza + PDF + Embeddings* + Estadística → **Make** Orquesta y envía notificaciones

*Embeddings vía HTTP desde Apps Script

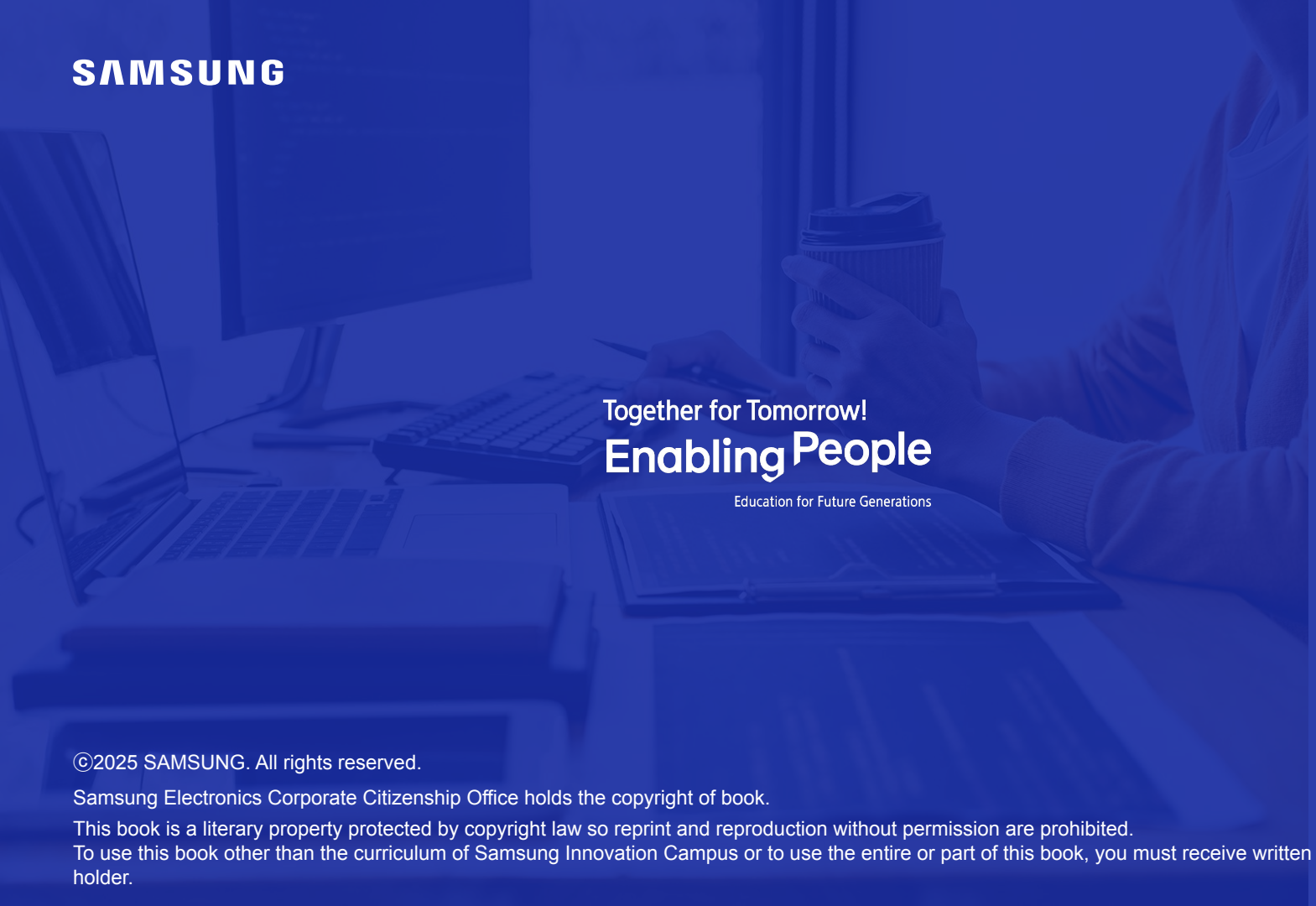
¿POR QUÉ ESTA ARQUITECTURA?

- **Separación de responsabilidades:** Make orquesta, aiserver calcula.
- **Escalabilidad:** el cómputo pesado vive fuera de Make.
- **Reutilización:** el mismo aiserver puede ser usado por otros flujos.
- **Mantenibilidad:** lógica compleja versionada en un solo lugar.
- **Costo-efectiva:** usamos lo justo en Make.

TECNOLOGÍAS CLAVE

Apify	para scraping
HTTP	para integrar servicios externos
aiserver	ETL + Embeddings + Stats
Google Sheets / Drive	para guardar y analizar
Make	como orquestador y conector

OBJETIVO FINAL: equivalente a nuestro **books_terror_clean.csv** listo para análisis y visualización.



ⓒ2025 SAMSUNG. All rights reserved.

Samsung Electronics Corporate Citizenship Office holds the copyright of book.

This book is a literary property protected by copyright law so reprint and reproduction without permission are prohibited.
To use this book other than the curriculum of Samsung Innovation Campus or to use the entire or part of this book, you must receive written consent from copyright holder.